

Nash Equilibrium on Rent Division Game

Game Theory - Project Report

April 2026

Abstract

This report presents manual implementation of the deep meaning "Nash Equilibrium" [2] terminology. The goal of this project is not only to understand the theory, but also to introduce the people behind this science - those whose hard work built its foundation [1]. Furthermore, it aims to demonstrate how these concepts work in practice by solving problems manually and implementing them through code applications. This report was prepared with the assistance of an AI writing tool [5].

Keywords: Nash equilibrium, Buckward Induction, IDSDS, IDWDS, Payoff, Pythonimplementation.

Contents

1	Introduction	2
1.1	Who is John Nash?	2
2	Preliminaries	2
2.1	Nash equilibrium	2
2.2	Nash equilibrium for "i" players	3
2.3	Nash equilibrium for "2" players	3
2.4	IDSDS / IDWDS in Nash equilibrium	3
3	Rent Division scenario	4
3.1	Creation of the settings	4
3.2	Assign valuations	4
3.3	Preferences	4
3.4	Divide the rent	5
3.5	Strategy Tables	5
3.6	Nash Equilibrium Identification	6
3.7	Mathematical Proof of Stability	6
4	Program Code	7
4.1	Code Explanation: Input Data (Lines 1–7)	7
4.2	Strategic Analysis and Nash Equilibrium (Lines 9–24)	7
4.3	Execution and Output Visualization (Lines 26–45)	9

5	Conclusion	9
6	Contribution	9

1 Introduction

1.1 Who is John Nash?

In 1944, together with economist Oskar Morgenstern, he (1928-2015) published the work "**Theory of Games and Economic Behavior**." As a schoolboy, young John did not like mathematics at all. Yet, at the age of 21, he wrote a paper for which he would receive the **Nobel Prize** in Economics 45 years later. Nash's idea was both simple and brilliant. The scientist proved that there are many cases where the sum of wins and losses is far from zero. Outcomes are possible where the sum is below zero (everyone loses), and the sum can also be above zero (the majority wins).



John Nash

The problems that game theory specialists worked on in the **1950s** and **1960s** concerned nuclear deterrence and the arms race. John Nash made Adam Smith cough, proving the fallacy of his assertion that competition is the main fuel of the market. The mathematician showed that the strategy "**everyone in the group does what is best for themselves**" ultimately implies only one winner from the entire group. Moreover, an outcome is possible in which no one becomes a winner. [1]

2 Preliminaries

2.1 Nash equilibrium

Nash equilibrium - is defined as a set of strategies for all players in a game such that each player's choice is a best response to the choices of the other players, meaning no player can profit by unilaterally changing their strategy.[2]

2.2 Nash equilibrium for "i" players

Mathematically, a Nash equilibrium is a strategy profile $(s_1^*, s_2^*, \dots, s_n^*)$ such that for every player i :

$$\pi_i(s_i^*, s_{-i}^*) \geq \pi_i(s_i, s_{-i}^*) \quad \forall s_i \in S_i$$

[2]

2.3 Nash equilibrium for "2" players

Given an ordinal game in strategic form with **two players** [3], a strategy profile $s^* = (s_1^*, s_2^*) \in S_1 \times S_2$ is a Nash equilibrium if the following **two conditions** are satisfied:

1. For every $s_1 \in S_1$:

$$\pi_1(s_1^*, s_2^*) \geq \pi_1(s_1, s_2^*)$$

(or stated in terms of outcomes and preferences: $f(s_1^*, s_2^*)_1 f(s_1, s_2^*)$).

2. For every $s_2 \in S_2$:

$$\pi_2(s_1^*, s_2^*) \geq \pi_2(s_1^*, s_2)$$

(or, $f(s_1^*, s_2^*)_2 f(s_1^*, s_2)$). [3] (**Definition 2.6.1**)

2.4 IDSDS / IDWDS in Nash equilibrium

IDSDS(G) $\subseteq$ S(G): Surviving strategies after iterated deletion of strictly dominated strategies.

IDWDS(G) $\subseteq$ S(G): Surviving strategies after iterated deletion of weakly dominated strategies. [3],

Every strategy profile from IDSDS is always a Nash equilibrium:

$$NE(G) \supseteq IDSDS(G)$$

Some Nash equilibria may not survive IDWDS.

G be an ordinal strategic-form game and $s \in S(G)$ a strategy profile in G .

(A) If $IDSDS(G) = \{s\}$, then s is a strict Nash equilibrium.

(B) If $IDWDS(G) = \{s\}$, then s is a Nash equilibrium. [3],

3 Rent Division scenario

3.1 Creation of the settings

Players and Contract

Three students "Malika, Agila, and Madina" sign a contract with the landlord Mustafa to rent a three-room apartment. The total rent is 300,000 .

Room Characteristics

The three rooms differ in size, view, and amenities:

- **Room 1:** Large, with a window and balcony.
- **Room 2:** Medium, with a perfect Wi-Fi connection.
- **Room 3:** Small, with an individual kitchen.

3.2 Assign valuations

Each student assigns a private value to each room (how much they think the room is worth).

Student	Room 1	Room 2	Room 3
Malika	150,000	100,000	50,000
Agila	120,000	140,000	40,000
Madina	130,000	60,000	110,000

Table 1: Private valuations

3.3 Preferences

Based on their private (see Table 1) valuations, the preference orderings of the students are as follows:

- **Malika:** Room 1 $\succ$ Room 2 $\succ$ Room 3 (She strongly prefers the large room with a balcony.)
- **Agila:** Room 2 $\succ$ Room 1 $\succ$ Room 3 (She values the perfect Wi-Fi connection more than the balcony.)
- **Madina:** Room 1 $\succ$ Room 3 $\succ$ Room 2 (She prefers the large room but would also accept the small room if necessary.)

3.4 Divide the rent

To achieve a stable allocation where no student wishes to switch rooms, the landlord Mustafa sets the following rents:

- Room 1: 120,000
- Room 2: 100,000
- Room 3: 80,000

These rents satisfy the budget constraint: $120,000 + 100,000 + 80,000 = 300,000$ [5]

3.5 Strategy Tables

$$Utility = Valuation - Rent$$

Table 2: Madina selects Room 1 (Utility: 10)

Malika \ Agila	Room 1	Room 2	Room 3
Room 1	(30, 0, 10)	(30, 40, 10)	(30, -40, 10)
Room 2	(0, 0, 10)	(0, 40, 10)	(0, -40, 10)
Room 3	(-30, 0, 10)	(-30, 40, 10)	(-30, -40, 10)

Table 3: Madina selects Room 2 (Utility: -40)

Malika \ Agila	Room 1	Room 2	Room 3
Room 1	(30, 0, -40)	(30, 40, -40)	(30, -40, -40)
Room 2	(0, 0, -40)	(0, 40, -40)	(0, -40, -40)
Room 3	(-30, 0, -40)	(-30, 40, -40)	(-30, -40, -40)

Table 4: Madina selects Room 3 (Utility: 30)

Malika \ Agila	Room 1	Room 2	Room 3
Room 1	(30, 0, 30)	(30, 40, 30)	(30, -40, 30)
Room 2	(0, 0, 30)	(0, 40, 30)	(0, -40, 30)
Room 3	(-30, 0, 30)	(-30, 40, 30)	(-30, -40, 30)

[5] by examples [3]

3.6 Nash Equilibrium Identification

We identify the **Nash Equilibrium** in **Table 4**, where Madina selects Room 3. The equilibrium occurs at the intersection of **Malika: Room 1** and **Agila: Room 2**.

Payoff vector:

$$(U_{Malika}, U_{Agila}, U_{Madina}) = (30, 40, 30)$$

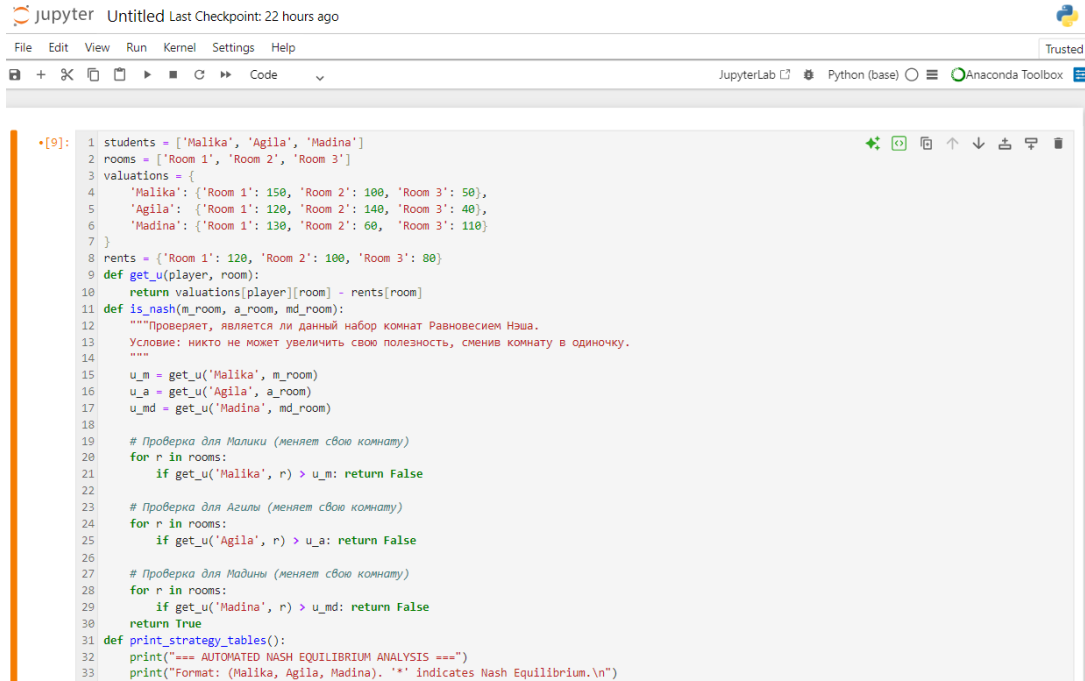
3.7 Mathematical Proof of Stability

We prove this by analyzing the incentives for each player:

- **Malika** stays in **Room 1**: She gets **30**. If she moves, she gets **0** or less.
- **Agila** stays in **Room 2**: She gets **40**. If she moves, she gets **0** or less.
- **Madina** stays in **Room 3**: She gets **30**. If she moves, she gets **10** or less.

Conclusion: Since no player can strictly increase their utility by changing their room choice while the others keep theirs unchanged (**Room 1, Room 2, Room 3**) is a stable **Nash Equilibrium**. [\[3\]](#)

4 Program Code



```
[9]: 1 students = ['Malika', 'Agila', 'Madina']
2 rooms = ['Room 1', 'Room 2', 'Room 3']
3 valuations = {
4     'Malika': {'Room 1': 150, 'Room 2': 100, 'Room 3': 50},
5     'Agila': {'Room 1': 120, 'Room 2': 140, 'Room 3': 40},
6     'Madina': {'Room 1': 130, 'Room 2': 60, 'Room 3': 110}
7 }
8 rents = {'Room 1': 120, 'Room 2': 100, 'Room 3': 80}
9 def get_u(player, room):
10     return valuations[player][room] - rents[room]
11 def is_nash(m_room, a_room, md_room):
12     """Проверит, является ли данный набор комнат Равновесием Нэша.
13     Условие: никто не может увеличить свою полезность, сменив комнату в одиночку.
14     """
15     u_m = get_u('Malika', m_room)
16     u_a = get_u('Agila', a_room)
17     u_md = get_u('Madina', md_room)
18
19     # Проверка для Малики (меняет свою комнату)
20     for r in rooms:
21         if get_u('Malika', r) > u_m: return False
22
23     # Проверка для Агилы (меняет свою комнату)
24     for r in rooms:
25         if get_u('Agila', r) > u_a: return False
26
27     # Проверка для Мадины (меняет свою комнату)
28     for r in rooms:
29         if get_u('Madina', r) > u_md: return False
30     return True
31 def print_strategy_tables():
32     print("=== AUTOMATED NASH EQUILIBRIUM ANALYSIS ===")
33     print("Format: (Malika, Agila, Madina). '*' indicates Nash Equilibrium.\n")
```

Figure 1: Python Implementation: Input and Core Logic [5]

4.1 Code Explanation: Input Data (Lines 1–7)

- **Lines 1–2 (Participants):** We define the set of players $N = \{Malika, Agila, Madina\}$ and the set of available strategies (rooms) $S = \{Room1, Room2, Room3\}$.
- **Lines 4–6 (Valuations):** This dictionary stores the private values (V_i) each student assigns to each room. For example, Malika values Room 1 at 150 units, reflecting her high preference for that specific space.
- **Line 7 (Rent Prices):** Here, we set the fixed prices (P_j) for each room as determined by Mustafa. These prices are crucial because they directly affect the net utility of each player.

4.2 Strategic Analysis and Nash Equilibrium (Lines 9–24)

This section of the code implements the core principles of game theory to find a stable solution:

- **Utility Calculation (Lines 9–10):** The function `get_u` represents the payoff function $\pi_i(s_i, s_{-i})$. It calculates the "net gain" for each student by subtracting the rent from their valuation. This value is what each player seeks to maximize.
- **Nash Equilibrium Condition (Lines 12–24):** The `is_nash` function is the heart of the strategic analysis. It verifies if a given room assignment is a Nash Equilibrium by checking for **unilateral deviations**:
 1. It calculates the current utility for Malika, Agila, and Madina.
 2. It then simulates what would happen if *only one* player changed their room.
 3. If any player can find a room that gives them a strictly higher utility than their current one, the function returns **False**, meaning the current state is not stable.

If the function returns **True**, it confirms that every player is playing their **Best Response** to the strategies of others, satisfying the condition $\pi_i(s_i^*, s_{-i}^*) \geq \pi_i(s_i, s_{-i}^*)$.

```

34 for md_choice in rooms:
35     print(f"--- Table: Madina chooses {md_choice} ---")
36     print(f"{'Malika \ Agila':<15} | {'Room 1':<12} | {'Room 2':<12} | {'Room 3':<12}")
37     print("-" * 65)
38     for m_choice in rooms:
39         row_cells = []
40         for a_choice in rooms:
41             p1, p2, p3 = get_u('Malika', m_choice), get_u('Agila', a_choice), get_u('Madina', md_choice)
42             payoff_str = f"({p1},{p2},{p3})"
43
44             if is_nash(m_choice, a_choice, md_choice):
45                 row_cells.append(f"{payoff_str:<10}")
46             else:
47                 row_cells.append(f"{payoff_str:<12}")
48
49         print(f"[m_choice:<15] | {' | '.join(row_cells)}")
50     print("\n")
51
52 if __name__ == "__main__":
53     print_strategy_tables()

```

=== AUTOMATED NASH EQUILIBRIUM ANALYSIS ===
Format: (Malika, Agila, Madina). '*' indicates Nash Equilibrium.

--- Table: Madina chooses Room 1 ---

Malika \ Agila	Room 1	Room 2	Room 3
Room 1	(30,0,10)	(30,40,10)	(30,-40,10)
Room 2	(0,0,10)	(0,40,10)	(0,-40,10)
Room 3	(-30,0,10)	(-30,40,10)	(-30,-40,10)

--- Table: Madina chooses Room 2 ---

Malika \ Agila	Room 1	Room 2	Room 3
Room 1	(30,0,-40)	(30,40,-40)	(30,-40,-40)
Room 2	(0,0,-40)	(0,40,-40)	(0,-40,-40)
Room 3	(-30,0,-40)	(-30,40,-40)	(-30,-40,-40)

--- Table: Madina chooses Room 3 ---

Malika \ Agila	Room 1	Room 2	Room 3
Room 1	(30,0,30)	*(30,40,30)*	(30,-40,30)
Room 2	(0,0,30)	(0,40,30)	(0,-40,30)
Room 3	(-30,0,30)	(-30,40,30)	(-30,-40,30)

Figure 2: Python Implementation: Strategy Matrices and Output [5]

4.3 Execution and Output Visualization (Lines 26–45)

The final part of the implementation is responsible for generating the strategy matrices and identifying the stable outcome:

- **Nested Strategy Tables (Lines 26–41):** Since we have three players, the code generates three separate 3x3 tables. Each table represents a fixed choice by Madina (Room 1, 2, or 3), allowing us to visualize the payoffs for Malika and Agila in a 2D format.
- **Automated Identification:** Within these tables, the program uses the `is_nash` function to automatically mark the Equilibrium. If a cell satisfies the Nash condition, it is highlighted with asterisks (e.g., `*(30, 40, 30)*`).
- **Final Result:** The output confirms that the only stable strategy profile is:
 - **Malika:** Room 1 (Utility: 30)
 - **Agila:** Room 2 (Utility: 40)
 - **Madina:** Room 3 (Utility: 30)

5 Conclusion

*"The best result comes from everyone in the group doing what is best for himself **and** the group."*

— John Nash[\[1\]](#)

In this project, we explored the work of John Nash, the creator of the "Nash Equilibrium" terminology[\[2\]](#). We have demonstrated how this theoretical framework can be applied to a real-world scenario: the fair division of rent among three roommates. By calculating the utilities for Malika, Agila, and Madina, we identified a stable strategy profile where no player has an incentive to change their room choice unilaterally.

The most challenging part of the implementation was developing the Nash Equilibrium function[\[2\]](#) from scratch. Since Python does not have a built-in library for this specific game-theoretic analysis, we manually implemented the logic to check for unilateral deviations, ensuring the mathematical stability of the final room assignment.

Our project follows Nash's[\[1\]](#) core principle by finding a room distribution where everyone is satisfied and the "community" of roommates remains stable. This proves that game theory is not just an abstract mathematical concept, but a powerful tool for achieving balance in everyday social interactions.

6 Contribution

Table 2: Individual team contributions

Team Member	Contribution
Uralina Agila/Rakhmatulla Mustafa	Prepared the project presentation slides, including visual graphs and tables, and led the team demonstration.
Orazalinova Malika	Co-authored the written report: drafted the Introduction, Preliminaries, Conclusion sections; Program Description and Code sections; structured the $\text{\LaTeX}$ document. verified all code listings for accuracy; compiled the bibliography.
Madina Abenova	Designed and implemented the full Python source code, including creating Nash function, and strategy profile, Validated correctness.

References

- [1] John Nash’s short biography. URL: <https://www.techinsider.ru/science/1670853-kak-priiti-k-nailuchshemu-ishoduv-lyubom-dele-ravnovesie-nesha-i-dilemma-zaklyuchennogo/>
- [2] Nash Equilibrium – an overview. ScienceDirect. URL: <https://www.sciencedirect.com/topics/economics-econometrics-and-finance/nash-equilibrium>
- [3] GAME THEORY. Third Edition. URL: <https://share.google/ap4ZH8VKSQXGCGrs2>
- [4] Backwards induction. URL: https://vknight.org/Year3_game_theory_course/Content/Chapter07 – Extensive_form_games_and_backwards_induction/
- [5] Google Gemini (Version 3.0), 2026. URL: <https://gemini.google.com>